

ACERTE

Estude junto. Passe junto.

Documentação Completa do Produto

Comunidade de vestibulandos de Medicina

Produto · Telas do MVP · Banco de Dados · Marca
Versão 1.0 — Junho de 2026

Sumário

1. Documento de Produto

Visão · Público · Problema · Solução · Proposta de valor · MVP · Futuras · Diferenciais · Jornada · Regras · Freemium · Roadmap 90 dias

2. Estrutura de Telas do MVP

Home · Login · Cadastro · Dashboard · Biblioteca · Vestibular · Material · Upload · Feed · Perfil · Ranking

3. Modelagem de Banco de Dados (Supabase/PostgreSQL)

Tabelas · Relacionamentos · Segurança · SQL · RLS

4. Textos Oficiais da Marca

Slogan · Home · Botões · Cadastro · Login · Upload · Ranking · Bio · Posts de lançamento

1. Documento de Produto

1.1 Visão Geral

O Acerte é uma plataforma SaaS que combina rede social profissional (LinkedIn), repositório colaborativo de conhecimento (GitHub) e biblioteca de estudos, focada exclusivamente em quem quer passar em Medicina.

O estudante constrói um perfil público que documenta sua jornada de preparação enquanto acessa um acervo organizado de provas, resumos e simulados por vestibular.

Acerte é onde o vestibulando de Medicina estuda, compartilha o que aprende e constrói reputação antes mesmo de entrar na faculdade.

Insight estratégico: o conteúdo é gerado e curado pela própria comunidade (modelo GitHub), e a reputação por contribuir é o que torna o perfil valioso (modelo LinkedIn). Isso cria efeito de rede e reduz o custo de produção de conteúdo.

1.2 Público-Alvo

- **Persona primária — O Vestibulando Intensivão:** 16–20 anos, em cursinho, mira públicas (USP, UNICAMP, UFMG, UFRJ, UNESP) e particulares de elite (Einstein, Santa Casa, PUCs). Estuda 6–10h/dia.
- **Persona secundária — O Veterano/Reaplicador:** já passou ou está na faculdade; quer ajudar, construir autoridade e monetizar. Fonte-chave de conteúdo de qualidade.
- **Persona terciária — O Calouro de 1ª tentativa:** ensino médio, menor intensidade. Funil de longo prazo.

Foco deliberado em Medicina: nicho com altíssima dor, alta disposição a pagar e comunidade fortíssima.

1.3 Problema

1. Conteúdo fragmentado e desorganizado (sites lentos, WhatsApp, drives, PDFs piratas).
2. Falta de curadoria e confiança: não se sabe qual resumo é bom ou atualizado.
3. Estudo solitário e desmotivante numa preparação longa (1–3 anos).
4. Quem produz bom material não é reconhecido.
5. Sem visão de progresso e evolução.

1.4 Solução

Quatro pilares integrados:

Pilar	O que faz	Inspiração
Biblioteca estruturada	Provas, gabaritos, simulados e resumos organizados por vestibular, ano, matéria e tema	GitHub
Colaboração & UGC	Qualquer estudante faz upload; a comunidade cura via curtidas, salvamentos e comentários	GitHub
Reputação & perfil	Contribuir gera pontos e um perfil público (currículo de estudo)	LinkedIn
Comunidade & feed	Feed social com materiais, dúvidas, conquistas e ranking	LinkedIn/Instagram

1.5 Proposta de Valor

Para o vestibulando: “Tudo o que você precisa para passar em Medicina, organizado em um só lugar — e validado por quem já estuda como você.”

Para o contribuidor: “Transforme seus resumos em reputação. Construa autoridade e seja reconhecido por milhares de futuros médicos.”

Diferencial vs. concorrentes (cursinhos, Estratégia MED, Sanar, Aristo): eles vendem conteúdo proprietário caro; o Acerte oferece acervo colaborativo + camada social + reputação, com lock-in via rede e perfil.

1.6 Funcionalidades do MVP

Objetivo: provar o loop conteúdo → curadoria → reputação → engajamento com 1–3 vestibulares-âncora (ENEM + FUVEST + UNICAMP).

- **Conta & Perfil:** cadastro/login (e-mail + Google); perfil público com bio, vestibular-alvo, materiais, reputação e badges.
- **Biblioteca:** navegação por vestibular → matéria → tema → ano; provas + gabaritos; resumos; busca e filtros.
- **Upload:** PDF/imagem com metadados obrigatórios e moderação leve.
- **Interações:** curtir, salvar, comentar; biblioteca pessoal de salvos.
- **Feed:** cronológico/relevância de materiais e atividades.
- **Reputação & Ranking:** pontos por contribuição; ranking geral e por vestibular.
- **Simulados (v1):** banco de questões objetivas com correção automática.

Cortes do MVP: sem IA, sem pagamento, sem mensagens diretas. Foco em validar o motor de conteúdo + comunidade.

1.7 Funcionalidades Futuras

- **Fase 2 — Engajamento:** mensagens diretas, grupos de estudo, notificações, trilhas, estatísticas de desempenho.

- **Fase 3 — IA:** plano de estudos personalizado, resumo automático, correção de redação por IA, recomendações.
- **Fase 4 — Ecossistema:** marketplace de mentorias, materiais premium, selos verificados, expansão para outros cursos.

1.8 Diferenciais

1. Reputação como moeda: contribuir gera status público e perfil de valor.
2. Foco vertical em Medicina: comunidade e conteúdo 100% específicos.
3. Curadoria social: o melhor conteúdo emerge da comunidade.
4. Custo marginal de conteúdo baixo (UGC), escalável.
5. Perfil público como ativo de longo prazo, retém além da aprovação.
6. IA futura como camada premium defensável e monetizável.

1.9 Jornada do Usuário

Aquisição → Ativação → Engajamento → Contribuição → Retenção/Receita

1. Descoberta via Instagram/TikTok, indicação ou busca.
2. Cadastro e escolha do vestibular-alvo.
3. Primeiro valor (Aha!): encontra/salva material relevante em < 2 min.
4. Engajamento: feed, curtidas, salvamentos, simulado.
5. Contribuição: primeiro upload → reputação → ranking.
6. Hábito: retorno diário pelo feed e progresso.
7. Conversão premium ao atingir limites do free.
8. Pós-aprovação: vira veterano/mentor e alimenta o ciclo.

Métrica-Aha: salvar ou baixar 3 materiais na 1ª sessão.

1.10 Regras de Negócio

- **Upload:** metadados obrigatórios; fila de moderação; 3+ reports válidos despublicam para revisão.
- **Direitos autorais:** provas oficiais públicas; material de terceiros protegido é proibido (takedown); resumos autorais.
- **Reputação (exemplo):** upload aprovado +10; curtida +2; salvamento +5; comentário +1; plágio -20.
- **Anti-fraude:** curtidas de contas novas valem menos; detecção de auto-curtida e contas duplicadas.
- **Ranking:** atualizado periodicamente, segmentado por vestibular, com decaimento temporal.
- **Perfil:** público por padrão; pode ser privado; badges por marcos.
- **LGPD/ECA:** público majoritariamente menor de idade — consentimento parental e conformidade.

1.11 Modelo Freemium / Premium

Recurso	Free	Premium (Acerte Pro)
Biblioteca (provas/resumos)	Total	Total
Curtir / salvar / comentar	Sim	Sim
Upload + reputação + ranking	Sim	Sim
Simulados	Limitado (3/mês)	Ilimitados + análise
Estatísticas de desempenho	Básicas	Avançadas
Anúncios	Sim (leves)	Sem anúncios
Plano de estudos com IA	Não	Sim
Resumo automático por IA	Não	Sim
Correção de redação por IA	Não	Sim (núcleo do valor pago)

Precificação sugerida: R\$ 19–29/mês ou R\$ 149–199/ano, com teste grátis de 7 dias. Princípio: nunca colocar o acervo colaborativo atrás do paywall.

1.12 Roadmap de 90 Dias

Período	Foco	Entregas principais
Dias 1–30	Fundação & Seed	Auth, perfil, biblioteca, upload; seed de provas (ENEM/FUVEST/UNICAMP) + 50 resumos; 10–20 veteranos fundadores.
Dias 31–60	Loop Social & Beta	Feed, curtidas, salvos, comentários, reputação, ranking, simulados v1; beta fechado (100–300 estudantes).
Dias 61–90	Lançamento & Growth	Polimento, indicação/viral, lançamento público com criadores, paywall leve (limite de simulados).

KPIs do trimestre: Retenção D7 $\geq$ 25–30%; $\geq$ 10% dos ativos fazem upload; $\geq$ 500–1.000 materiais; $\geq$ 3–5% de conversão para Pro.

2. Estrutura de Telas do MVP

Convenções: **[Botão]** = ação; **{dados}** = backend; **(L)** = exige login.

2.1 Home (/)

Objetivo: Converter visitante em cadastro e explicar a proposta de valor.

Componentes: Header público; Hero (headline + CTA); Como funciona (3 passos); Cards de vestibulares; Prova social; Footer.

Botões: [Criar conta grátis] · [Entrar] · [Explorar biblioteca]

Informações exibidas: Headline de valor; métricas agregadas; vestibulares ativos; depoimentos.

Ações do usuário: Ir para cadastro/login; navegar para biblioteca; clicar em vestibular.

Tudo para passar em Medicina — organizado e validado por quem estuda como você.

2.2 Login (/login)

Objetivo: Autenticar usuário existente rapidamente.

Componentes: Card de login; campos e-mail/senha; botão Google; links recuperar/cadastro.

Botões: [Entrar] · [Continuar com Google] · Esqueci minha senha · Criar conta

Informações exibidas: Formulário; erros inline; loading.

Ações do usuário: Enviar credenciais; login social; recuperar senha.

Que bom te ver de volta. Sua aprovação está mais perto a cada dia.

2.3 Cadastro (/cadastro)

Objetivo: Criar conta com atrito mínimo e capturar o vestibular-alvo.

Componentes: Card em 2 passos; dados ou Google; seleção de vestibular; tipo de usuário; aceite de termos.

Botões: [Criar conta] · [Continuar com Google] · [Finalizar] · Já tenho conta

Informações exibidas: Campos; validação inline; lista de vestibulares.

Ações do usuário: Preencher dados; escolher vestibular; aceitar termos.

Comece sua jornada até a Medicina. Qual vestibular você quer conquistar?

2.4 Dashboard (/dashboard) (L)

Objetivo: Tela inicial pós-login: próximo passo, progresso e atalhos.

Componentes: Saudação + reputação; cards de atalho; widget de progresso; recomendados; resumo do feed.

Botões: [Fazer upload] · [Ir para a biblioteca] · [Fazer simulado] · [Ver meu perfil]

Informações exibidas: {nome}; {reputacao}; {posicao_ranking}; {recomendados}; {atividade_recente}.

Ações do usuário: Navegar; abrir material; iniciar simulado; upload.

Olá, {nome}. Vamos mais perto da sua aprovação em {vestibular}?

2.5 Biblioteca (/biblioteca)

Objetivo: Descoberta e navegação do acervo por vestibular, matéria, tema, tipo e ano.

Componentes: Barra de busca; filtros; grid de cards; vestibulares em destaque; paginação.

Botões: [Buscar] · Chips de filtro · [Limpar filtros] · [Salvar] (L) · [Abrir] · [Fazer upload] (L)

Informações exibidas: Título, tipo, vestibular, matéria, ano; autor; curtidas/salvamentos/downloads.

Ações do usuário: Buscar, filtrar, ordenar; abrir/salvar material.

Busque por prova, resumo, matéria ou tema...

2.6 Página de Vestibular (/vestibular/:slug)

Objetivo: Hub de um vestibular, agregando todo o conteúdo e contexto do exame.

Componentes: Header do vestibular; abas (Provas, Resumos, Simulados, Top contribuidores); filtro matéria/ano; bloco Sobre.

Botões: Abas · [Seguir vestibular] (L) · [Abrir] · [Salvar] · [Contribuir] (L)

Informações exibidas: {nome}; {descricao}; {total_materiais}; materiais por categoria; ranking de contribuidores.

Ações do usuário: Alternar abas; filtrar; abrir/salvar; seguir vestibular.

{Nome do Vestibular} — {total} materiais, {provas} provas, {contribuidores} contribuidores.

2.7 Página de Material (/material/:id)

Objetivo: Exibir o material e concentrar interações sociais.

Componentes: Cabeçalho + badges; visualizador de PDF; bloco do autor; barra de ações; comentários; relacionados.

Botões: [Curtir] (L) · [Salvar] (L) · [Baixar] (L) · [Compartilhar] · [Comentar] (L) · [Denunciar] · [Seguir] (L)

Informações exibidas: {titulo}; {autor} + reputação; métricas; {comentarios}; relacionados.

Ações do usuário: Visualizar, baixar, curtir, salvar, comentar, denunciar, seguir.

Adicione um comentário... | Seja o primeiro a comentar.

2.8 Upload de Material (/upload) (L)

Objetivo: Permitir contribuição com metadados padronizados e qualidade.

Componentes: Área de upload (drag-and-drop); campos obrigatórios; descrição; declaração de autoria; preview.

Botões: [Selecionar arquivo] · [Publicar material] · [Salvar rascunho] · [Cancelar]

Informações exibidas: Status do upload; validações; aviso de moderação; preview.

Ações do usuário: Anexar; preencher metadados; declarar autoria; publicar.

Compartilhe seu material. Ajude a comunidade e ganhe reputação a cada contribuição.

2.9 Feed da Comunidade (/feed)

Objetivo: Engajamento diário: atividade da comunidade e hábito de retorno.

Componentes: Filtros (Para você / Seguindo / Recentes); caixa de post (L); cards de atividade; sidebar de sugestões.

Botões: Abas de filtro · [Publicar] (L) · [Curtir] · [Comentar] · [Salvar] · [Seguir]

Informações exibidas: {atividades}; sugestões de usuários; destaques da semana.

Ações do usuário: Filtrar, postar, curtir/comentar/salvar, seguir.

Compartilhe uma dica, dúvida ou conquista...

2.10 Perfil do Usuário (/u/:username)

Objetivo: Currículo de estudos público: reputação, contribuições e progresso.

Componentes: Header do perfil; painel de reputação; estatísticas; abas (Materiais, Salvos, Atividade).

Botões: [Seguir]/[Seguindo] (L) · [Editar perfil] · [Compartilhar perfil] · Abas

Informações exibidas: {nome}; {bio}; {vestibular}; {reputacao}; {badges}; {materiais}; seguidores.

Ações do usuário: Seguir; ver materiais; compartilhar; editar (próprio).

Mira: {vestibular} - {reputacao} pts - #{posicao} no ranking.

2.11 Ranking (/ranking)

Objetivo: Gamificar contribuição e engajamento; reconhecimento.

Componentes: Filtros (Geral/Por vestibular + período); pódio top 3; tabela ranqueada; destaque da posição do usuário.

Botões: Filtros · Avatares → perfil · [Subir no ranking] → upload

Informações exibidas: {ranking}; {minha_posicao}; top 3 em destaque.

Ações do usuário: Filtrar, abrir perfis, ir para upload.

Ranking de quem mais ajuda a comunidade. Você está em #{posicao}.

3. Modelagem de Banco de Dados (Supabase / PostgreSQL)

Usa auth.users (Supabase Auth) como base de identidade e tabelas no schema public. PDFs no Storage (bucket materials); a tabela materials guarda apenas o caminho.

3.1 Visão Geral das Tabelas

Tabela	Função
profiles	Dados públicos do usuário (extensão de auth.users)
vestibulares	Vestibulares (ENEM, FUVEST...)
faculdades	Faculdades/instituições
vestibular_faculdades	N:N entre vestibular e faculdade
material_categories	Categorias/tipos de material
subjects	Matérias (Biologia, Química...)
materials	Materiais enviados (metadados + ref. ao PDF)
posts	Posts do feed da comunidade
comments	Comentários (polimórfico: material/post)
likes	Curtidas (polimórfico: material/post/comment)
saves	Salvamentos de materiais
follows	Relação seguir usuário
reputation_events	Histórico de pontos (fonte da verdade)
reports	Denúncias de conteúdo

3.2 Relacionamentos

- auth.users 1—1 profiles
- profiles 1—N materials / posts / comments / likes / saves / follows / reputation_events
- vestibulares 1—N materials; vestibulares N—N faculdades (via vestibular_faculdades)
- material_categories 1—N materials; subjects 1—N materials
- materials 1—N comments / likes / saves / reports; posts 1—N comments / likes
- Curtidas e comentários são polimórficos via par (target_type, target_id)

3.3 Regras de Segurança (princípios)

1. RLS ligado em todas as tabelas do public.
2. Leitura pública de conteúdo aprovado (materials status='approved', perfis não privados).
3. Escrita só do dono: author_id/user_id = auth.uid().
4. Reputação imutável pelo cliente: gravada só por funções SECURITY DEFINER/triggers.
5. Moderação (aprovar material) restrita a role='admin'.
6. Storage privado: download de material aprovado; upload na pasta do próprio usuário.

3.4 SQL Inicial — Criação das Tabelas

```
create extension if not exists "pgcrypto";

create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  username text unique not null,
  full_name text,
  bio text,
  avatar_url text,
  role text not null default 'vestibulando'
  check (role in ('vestibulando','veterano','admin')),
  target_vestibular_id uuid,
  reputation int not null default 0,
  is_private boolean not null default false,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.vestibulares (
  id uuid primary key default gen_random_uuid(),
  name text not null, slug text unique not null,
  description text, logo_url text,
  is_active boolean not null default true,
  created_at timestamptz not null default now()
);

create table public.faculdades (
  id uuid primary key default gen_random_uuid(),
  name text not null, slug text unique, state char(2),
  type text check (type in ('publica','particular')),
  logo_url text, created_at timestamptz not null default now()
);

create table public.vestibular_faculdades (
  vestibular_id uuid not null references public.vestibulares(id) on delete cascade,
  faculdade_id uuid not null references public.faculdades(id) on delete cascade,
  primary key (vestibular_id, faculdade_id)
);

alter table public.profiles
  add constraint profiles_target_vestibular_fk
  foreign key (target_vestibular_id) references public.vestibulares(id) on delete set null;

create table public.material_categories (
  id uuid primary key default gen_random_uuid(),
  name text not null, slug text unique not null
);

create table public.subjects (
  id uuid primary key default gen_random_uuid(),
  name text not null, slug text unique not null
);

create table public.materials (
  id uuid primary key default gen_random_uuid(),
  author_id uuid not null references public.profiles(id) on delete cascade,
  title text not null, description text,
  category_id uuid references public.material_categories(id),
  vestibular_id uuid references public.vestibulares(id),
  subject_id uuid references public.subjects(id),
  topic text, year int check (year between 1990 and 2100),
  file_path text not null, file_type text default 'pdf',
```

```

    status text not null default 'pending'
    check (status in ('pending','approved','rejected')),
    likes_count int not null default 0,
    saves_count int not null default 0,
    comments_count int not null default 0,
    downloads_count int not null default 0,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now()
);
create index on public.materials (vestibular_id);
create index on public.materials (subject_id);
create index on public.materials (status);
create index on public.materials (author_id);

create table public.posts (
    id uuid primary key default gen_random_uuid(),
    author_id uuid not null references public.profiles(id) on delete cascade,
    content text not null,
    material_id uuid references public.materials(id) on delete set null,
    likes_count int not null default 0,
    comments_count int not null default 0,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now()
);

create table public.comments (
    id uuid primary key default gen_random_uuid(),
    author_id uuid not null references public.profiles(id) on delete cascade,
    target_type text not null check (target_type in ('material','post')),
    target_id uuid not null,
    parent_id uuid references public.comments(id) on delete cascade,
    content text not null, likes_count int not null default 0,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now()
);
create index on public.comments (target_type, target_id);

create table public.likes (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references public.profiles(id) on delete cascade,
    target_type text not null check (target_type in ('material','post','comment')),
    target_id uuid not null,
    created_at timestamptz not null default now(),
    unique (user_id, target_type, target_id)
);

create table public.saves (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references public.profiles(id) on delete cascade,
    material_id uuid not null references public.materials(id) on delete cascade,
    created_at timestamptz not null default now(),
    unique (user_id, material_id)
);

create table public.follows (
    follower_id uuid not null references public.profiles(id) on delete cascade,
    following_id uuid not null references public.profiles(id) on delete cascade,
    created_at timestamptz not null default now(),
    primary key (follower_id, following_id),
    check (follower_id <> following_id)
);

create table public.reputation_events (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references public.profiles(id) on delete cascade,

```

```

type text not null, points int not null,
source_type text, source_id uuid,
created_at timestamptz not null default now()
);

create table public.reports (
  id uuid primary key default gen_random_uuid(),
  reporter_id uuid not null references public.profiles(id) on delete cascade,
  target_type text not null check (target_type in ('material','post','comment')),
  target_id uuid not null, reason text,
  status text not null default 'open'
  check (status in ('open','reviewed','dismissed')),
  created_at timestamptz not null default now()
);

```

3.5 Trigger — Criar profile no signup

```

create or replace function public.handle_new_user()
returns trigger language plpgsql security definer set search_path = public as $$
begin
  insert into public.profiles (id, username, full_name)
  values (
    new.id,
    coalesce(new.raw_user_meta_data->>'username', split_part(new.email,'@',1)),
    new.raw_user_meta_data->>'full_name'
  );
  return new;
end; $$;

create trigger on_auth_user_created
after insert on auth.users
for each row execute function public.handle_new_user();

```

3.6 Row Level Security (RLS)

```
-- Ativar RLS (em todas as tabelas)
alter table public.profiles enable row level security;
alter table public.materials enable row level security;
alter table public.posts enable row level security;
alter table public.comments enable row level security;
alter table public.likes enable row level security;
alter table public.saves enable row level security;
alter table public.follows enable row level security;
alter table public.reputation_events enable row level security;
alter table public.reports enable row level security;

create or replace function public.is_admin()
returns boolean language sql stable security definer set search_path = public as $$
    select exists(select 1 from public.profiles
                  where id = auth.uid() and role = 'admin');
$$;

-- PROFILES
create policy "read public profiles" on public.profiles for select
    using (is_private = false or id = auth.uid() or public.is_admin());
create policy "update own profile" on public.profiles for update
    using (id = auth.uid()) with check (id = auth.uid());

-- MATERIALS
create policy "read approved or own" on public.materials for select
    using (status = 'approved' or author_id = auth.uid() or public.is_admin());
create policy "insert own material" on public.materials for insert
    with check (author_id = auth.uid());
create policy "update own material" on public.materials for update
    using (author_id = auth.uid()) with check (author_id = auth.uid());
create policy "admin moderate" on public.materials for update
    using (public.is_admin()) with check (public.is_admin());
create policy "delete own material" on public.materials for delete
    using (author_id = auth.uid() or public.is_admin());

-- POSTS / COMMENTS (leitura pública, escrita do dono)
create policy "read posts" on public.posts for select using (true);
create policy "insert own post" on public.posts for insert
    with check (author_id = auth.uid());
create policy "read comments" on public.comments for select using (true);
create policy "insert own comment" on public.comments for insert
    with check (author_id = auth.uid());

-- LIKES
create policy "read likes" on public.likes for select using (true);
create policy "insert own like" on public.likes for insert
    with check (user_id = auth.uid());
create policy "delete own like" on public.likes for delete
    using (user_id = auth.uid());

-- SAVES (privado do dono)
create policy "read own saves" on public.saves for select
    using (user_id = auth.uid());
create policy "insert own save" on public.saves for insert
    with check (user_id = auth.uid());

-- REPUTATION (somente leitura; grava só via security definer)
create policy "read own reputation" on public.reputation_events for select
    using (user_id = auth.uid() or public.is_admin());
```


Observação: para impedir que o autor altere o campo status, adicione um trigger BEFORE UPDATE em materials que reverte status quando auth.uid() não for admin. A reputação deve ser gravada apenas por triggers/funções SECURITY DEFINER (nenhuma policy de INSERT para o cliente em reputation_events).

4. Textos Oficiais da Marca

Tom: jovem, motivador, confiável e focado na aprovação em Medicina.

4.1 Slogan Principal

Acerte. Da primeira tentativa à jaleco.

Alternativas: “Sua aprovação em Medicina começa aqui.” · “Estude junto. Passe junto.” · “O caminho mais curto até a Medicina.”

4.2 Subtítulo da Home

A comunidade dos vestibulandos de Medicina. Provas, simulados, resumos e materiais compartilhados por quem estuda como você — e por quem já passou.

4.3 Texto da Página Inicial

Headline: Tudo para passar em Medicina, em um só lugar.

Subheadline: Pare de caçar PDF em grupo de WhatsApp. No Acerte você encontra provas antigas, simulados e resumos organizados por vestibular — validados por uma comunidade que tem o mesmo objetivo que você: o jaleco branco.

- **Acesse o que importa.** Provas, gabaritos, simulados e resumos organizados por vestibular, matéria e tema.
- **Estude com a comunidade.** Curta, salve, comente e siga quem produz os melhores materiais.
- **Contribua e seja reconhecido.** Compartilhe seus resumos, suba no ranking e construa sua reputação.

Chamada final: A sua vaga em Medicina não vai esperar. E você? Crie sua conta gratuita e comece a estudar com a comunidade hoje.

4.4 Texto do Botão Principal

- **Primário:** “Começar agora — é grátis”
- **Hero:** “Criar minha conta grátis”
- **Secundário:** “Explorar a biblioteca”
- **Final da página:** “Quero minha aprovação”

4.5 Texto da Página de Cadastro

Título: Comece sua jornada até a Medicina

Subtítulo: Leva menos de 1 minuto. Sua aprovação não pode esperar.

Onboarding: Qual vestibular você vai conquistar? · Você é vestibulando ou já está na Medicina?

Boas-vindas: Bem-vindo(a) ao Acerte! Sua jornada rumo ao jaleco começa agora.

4.6 Texto da Página de Login

Título: Que bom te ver de volta

Subtítulo: Sua aprovação está mais perto a cada dia. Bora estudar?

Erro genérico: E-mail ou senha incorretos. Tenta de novo?

4.7 Texto da Página de Upload

Título: Compartilhe e ajude a próxima turma a passar

Subtítulo: Aquele resumo que te salvou pode salvar milhares de outros vestibulandos. Compartilhe, ganhe reputação e suba no ranking.

Sucesso: Material enviado! Assim que for revisado, ele vai aparecer para toda a comunidade — e os pontos são seus.

Quem ensina, aprende duas vezes. E ainda sobe no ranking.

4.8 Texto da Página de Ranking

Título: Ranking de quem mais ajuda a comunidade

Subtítulo: Aqui, quem compartilha conhecimento vira referência. Contribua, ganhe reputação e mostre que você está no caminho da aprovação.

Faixa do usuário: Você está em #{posição}. Compartilhe mais um material e suba de posição!

Os melhores médicos começaram ajudando os colegas a passar.

4.9 Bio para Instagram

 A comunidade dos vestibulandos de Medicina
 Provas · Simulados · Resumos · Materiais
 Estude junto. Passe junto.
 Crie sua conta grátis

4.10 Dez Ideias de Posts para Lançamento

1. Anúncio/teaser: “Chegou o lugar que faltava pra quem estuda pra Medicina.” (vídeo/carrossel)
2. A dor (relatável): POV de procurar prova antiga em 6 grupos de WhatsApp. (meme/Reels)
3. Prova social: +10.000 materiais, milhares de futuros médicos, um só objetivo. (carrossel)

4. Tutorial: como achar a prova da FUVEST 2023 resolvida em 10 segundos. (screen recording)
5. Depoimento aspiracional: estudava sozinha e travava; no Acerte encontrei foco. (card)
6. Convite a veteranos: vire Contribuidor Fundador e seja referência. (carrossel/stories)
7. Motivacional: cada resumo de hoje é um passo mais perto do jaleco. (card)
8. Funcionalidade Ranking: ajudar a comunidade te coloca no topo. (print + explicação)
9. Conteúdo de valor: as 5 matérias que mais caem e onde estudar de graça. (carrossel)
10. Urgência/CTA final: o Acerte está no ar e o cadastro é grátis. Link na bio. (Reels)

Bordão da marca: “Estude junto. Passe junto.” — símbolo emocional: o jaleco branco.